

AZ-204

Developing Solutions for Microsoft Azure

Guía de Puntos Clave para el Examen

v5.0 — Preparado por Max Carrión | Enriquecido con feedback MCT + SDK & CLI expandido

Dominio	Peso en el examen
Desarrollo de soluciones de proceso de Azure	25-30 %
Desarrollo para Azure Storage	15-20 %
Implementación de la seguridad de Azure	15-20 %
Supervisión, solución de problemas y optimización	5-10 %
Conexión y consumo de servicios de Azure y terceros	20-25 %

Puntuación mínima para aprobar: 700 / 1000

PARTE 1 — Desarrollo de Soluciones de Proceso (25-30%)

1.1 Azure Container Registry (ACR)

Niveles de servicio

Nivel	Descripción	Uso típico
Básico	Capacidades idénticas a Estándar/Premium pero almacenamiento y throughput limitados	Desarrollo y aprendizaje
Estándar	Mayor almacenamiento y throughput que Básico	Mayoría de escenarios producción
Premium	Mayor throughput + geo-replicación + zone redundancy + Private Link + confianza de contenido	Alta disponibilidad y escenarios distribuidos

✂ CLAVE

Solo **Premium** soporta geo-replicación, redundancia de zona y Private Link. Todas las imágenes se cifran en reposo (AES-256). No hay costo adicional por el cifrado.

Admin Account de ACR viene **DESHABILITADO** por defecto. Debes habilitarlo explícitamente si lo necesitas.

Almacenamiento y seguridad ACR

- **Cifrado en reposo** automático con AES-256 — no requiere configuración adicional
- **Geo-replicación** (solo Premium): protege ante fallos regionales, mejora latencia de pull/push
- **Redundancia de zona** (solo Premium): replica en mínimo 3 zonas de disponibilidad
- Almacenamiento regional: datos en la región del registro (excepción: **Brasil Sur y Asia SE** — siempre confinados a la región, no pueden replicarse)
- Recursos eliminados (repos, imágenes, etiquetas) **NO son recuperables**
- Soporta Helm charts e imágenes OCI además de imágenes Docker estándar

ACR Tasks — Automatización de compilaciones

- **Tarea rápida (Quick Task)**: compilar e insertar imagen on-demand sin Docker local
- **Tarea por actualización de código fuente**: webhook en GitHub/Azure DevOps activa la compilación
- **Tarea por actualización de imagen base**: recompila automáticamente cuando cambia la imagen padre
- **Tarea programada**: ejecución por temporizador (mantenimiento, pruebas periódicas)
- **Tarea de varios pasos (YAML)**: define flujos build → test → push con múltiples contenedores

📄 SDK / CÓDIGO

- `az acr build --registry <nombre> --image <imagen>:<tag> .` — envía contexto a ACR Tasks y compila (Quick Task)
- `az acr task create --registry <nombre> --name <task> --image <imagen>:<tag> --context <repo-git> --file Dockerfile --git-access-token <token>` — tarea por actualización de código
- `az acr build --platform Linux/arm64/v8 ...` — compilar para arquitectura ARM (ej. ATMs)
- `az acr build --platform Windows/amd64 ...` — compilar para Windows

1.2 Azure Container Instances (ACI) y Azure Container Apps (ACA)

Azure Container Instances (ACI)

- Ejecución de contenedores sin administrar VMs ni orquestadores
- Inicio rápido (segundos) — ideal para tareas cortas, CI, procesamiento por lotes
- **Grupos de contenedores:** múltiples contenedores en el mismo host (comparten red y almacenamiento)
- **Montaje de volúmenes:** Azure Files, repositorios Git, secretos, Empty directory
- **Políticas de reinicio:** Always / OnFailure / Never
- OS type: Linux (default) o Windows. DNS name label: <label>.<region>.azurecontainer.io

📖 SDK / CÓDIGO

- `az container create --resource-group <rg> --name <nombre> --image <imagen> --restart-policy OnFailure` — política de reinicio
- `az container create ... --environment-variables key=value --secure-environment-variables key=value` — variables seguras
- `az container create ... --azure-file-volume-account-name <cuenta> --azure-file-volume-account-key <clave> --azure-file-volume-share-name <share> --azure-file-volume-mount-path /mnt/datos` — montar Azure Files
- `az container show --resource-group <rg> --name <nombre>` — ver estado del contenedor
- `az container logs --resource-group <rg> --name <nombre>` — ver logs del contenedor

★ NOTA — Variables de Entorno Seguras (secureValue)

Para pasar cadenas de conexión sensibles (ej. credenciales de BD bancaria), **NUNCA uses la propiedad value normal**.

Usa la propiedad **secureValue**. Las variables con secureValue **no son visibles** en las propiedades del contenedor en el portal ni en la API REST.

Solo se puede acceder a su valor **desde dentro del contenedor**.

En YAML: { name: "DB_CONN", secureValue: "Server=banco-prod..." }

🚫 TRAMPA DEL EXAMEN

CRÍTICO — Los grupos de contenedores **SOLO** están disponibles en Linux. Windows solo permite 1 contenedor por grupo.

ACI NO tiene load balancer integrado. Para balanceo entre contenedores, usar ACA o AKS.

Port mapping NO se admite — los contenedores en un mismo grupo comparten la misma IP y espacio de puertos. Se comunican entre sí por **localhost**.

Volúmenes Azure Files en Linux: el contenedor **DEBE ejecutarse como root**. Limitados a compatibilidad CIFS.

Volúmenes en Windows: solo Azure Files mount. Linux soporta todos los tipos (Files, Git, secretos, Empty dir).

Azure Container Apps (ACA)

- Plataforma serverless para microservicios y apps en contenedores — abstrae Kubernetes
- **Escala horizontal basada en KEDA:** HTTP, colas, métricas custom. Puede escalar a **CERO** instancias
- **EXCEPCIÓN escala a cero:** ACA NO puede escalar a cero cuando la escala se basa en **métricas de CPU o memoria**
- **Ingress integrado** (HTTPS), revision management, split traffic (canary/blue-green)

- **Entornos:** unidad de aislamiento que agrupa múltiples Container Apps
- **Dapr:** sidecar para comunicación service-to-service, state, pub/sub. Puertos: **HTTP 3500 / gRPC 50001**

📁 SDK / CÓDIGO

- `az containerapp create --name <nombre> --resource-group <rg> --environment <env> --image <imagen> --target-port 80 --ingress external` — crear app con ingress público
- `az containerapp update --name <nombre> --resource-group <rg> --image <nueva-imagen>` — actualizar imagen (genera nueva revisión si hay cambios de revisión)
- `az containerapp revision list --name <nombre> --resource-group <rg>` — listar revisiones
- `az containerapp create ... --secrets "mi-secreto=valor" --env-vars "VAR=secretref:mi-secreto"` — pasar secreto a variable de entorno

★ NOTA — Gestión de Secretos en ACA

Los secretos en ACA tienen **ámbito de aplicación**, NO de revisión.

Si rotas la contraseña de un API bancario externo: cambiar el secreto **NO genera una nueva revisión** automáticamente.

Debes **reiniciar una revisión existente o implementar una nueva** para que tome el cambio.

TRAMPA: ACA **NO admite integración directa nativa** con Azure Key Vault en su configuración. Para usar KV, debes habilitar una Identidad Administrada y usar el **SDK de Key Vault dentro del código**.

ACA **no puede ejecutar contenedores privilegiados** (no permite acceso root a nivel host).

1.3 Azure App Service

Planes de App Service

Plan	Escalado	Característica clave
Free/Shared (F1, D1)	Una sola instancia	No soporta escalado automático
Basic (B1-B3)	MANUAL solamente	NO soporta escalado automático
Standard (S1-S3)	Automático basado en reglas	Deployment Slots disponibles
Premium V2/V3	Automático avanzado + Automatic Scaling	Instancias siempre listas, precalentadas, mayor rendimiento

⚡ CLAVE

El escalado automático requiere plan **Standard o superior**. F1 y D1 = 1 instancia. B1-B3 = manual. El examen intenta hacer elegir B1 para autoscale.

Escalado automático — Conceptos clave

- **Scale out/in (horizontal):** agrega/quita instancias — NO cambia CPU/RAM de cada instancia
- **Escalar según métricas:** CPU%, memoria%, longitud cola HTTP, longitud cola disco, entrada/salida de datos
- **Escalar según programación:** fija un conteo de instancias en horarios definidos
- **Período de recuperación (cooldown) mínimo:** 5 minutos entre eventos de escalado

- Lógica de combinación de reglas:
 - **Scale-out:** se dispara si se cumple CUALQUIERA de las reglas (OR)
 - **Scale-in:** se dispara solo si se cumplen TODAS las reglas (AND)
- **Oscilación (Flapping):** el motor previene bucles infinitos estimando el estado futuro. Si al quitar instancias la métrica volvería a disparar un scale-out, **omite la reducción**. Deja siempre un margen amplio entre umbrales
- **Autoscale vs Automatic Scaling:** En Premium V2/V3 existe "Automatic Scaling" — la plataforma gestiona la escala por tráfico HTTP sin configurar reglas, con instancias precalentadas garantizadas. Distinto al Autoscale clásico basado en métricas

📖 SDK / CÓDIGO

- `az webapp list-runtimes --os-type linux` — listar runtimes disponibles en Linux
- `az webapp up --name <nombre> --resource-group <rg> --runtime "DOTNET|8.0"` — deploy directo (crea app si no existe)
- `az webapp config appsettings set --resource-group <rg> --name <app> --settings key=value` — configurar settings
- `az webapp log tail --name <app> --resource-group <rg>` — streaming de logs en tiempo real
- `az webapp show --resource-group <rg> --name <app> --query outboundIpAddresses` — ver IPs de salida

⚡ CLAVE

Duración mínima para análisis: **5 minutos**. El intervalo de tiempo BASE es **1 minuto** (agregado sobre la duración configurada). Son dos conceptos distintos — el examen los confunde.

Métricas disponibles para escalado automático

Métrica	Qué indica
Porcentaje de CPU	Uso de CPU en todas las instancias
Porcentaje de memoria	Uso de memoria — riesgo de fallos si es alto
Longitud de cola de disco	Solicitudes de E/S pendientes — contención de disco
Longitud de cola HTTP	Solicitudes esperando procesamiento — errores HTTP 408 si es alta
Entrada de datos (bytes)	Bytes recibidos en todas las instancias
Salida de datos (bytes)	Bytes enviados desde todas las instancias

Deployment Slots (ranuras de implementación)

- Disponibles en planes **Standard y superior**
- Cada slot es una app independiente con su propia URL
- **Swap:** intercambia configuración entre slots (ej: staging → production)
- **Intercambio con Vista Previa (Multi-fase):** Aplica la config del slot destino (Producción) al slot origen (Staging) y **pausa el intercambio**. Permite probar cómo funciona la app con variables de producción antes de enrutar tráfico real

- **Configuraciones NO intercambiables (sticky):** Always On, configuración de escala, Identidades Administradas, restricciones IP, VNET integration, scheduler de WebJobs
- **Configuraciones SÍ intercambiables:** Application Settings y Connection Strings (SALVO que estén marcadas como "deployment slot setting")
- **Calentamiento personalizado:** usa la variable de entorno **WEBSITE_SWAP_WARMUP_PING_PATH** (ej. /healthcheck) para esperar HTTP 200 de esa ruta antes de finalizar el swap
- **Enrutamiento oculto:** configura tráfico en 0% y los testers pueden acceder con parámetro **?x-ms-routing-name=staging**
- La cookie **x-ms-routing-name** identifica el slot asignado al cliente

⚡ CLAVE

Durante un swap, las solicitudes al slot de destino se redirigen gradualmente — permite despliegues sin tiempo de inactividad.

Managed Identities NUNCA se intercambian en un swap, ni siquiera si no están marcadas como sticky. Esto es una trampa frecuente.

Auto Swap: NO disponible para Linux Web Apps ni Web App for Containers.

Configuración y settings clave

- **Application Settings:** pares clave-valor que sobrescriben appSettings/connectionStrings del código
- **Connection Strings:** configuradas por tipo (SQLServer, MySQL, PostgreSQL, Custom)
- **Always On:** mantiene la app activa, necesario para WebJobs continuos
- **ARR Affinity:** sesiones de afinidad con instancia específica — DESACTIVAR si la app es stateless
- **HTTPS Only:** fuerza redirección HTTP → HTTPS
- **Minimum TLS Version:** versión mínima TLS para conexiones entrantes
- **Configuración anidada en Linux:** reemplaza ":" por **doble guion bajo (__)** en el portal. Ej: ApplicationInsights__InstrumentationKey
- **Certificados privados:** archivo PFX con contraseña, cifrado Triple DES, clave privada mínimo **2048 bits**

🚫 TRAMPA DEL EXAMEN

Application Logging en sistema de archivos: es SOLO para depuración temporal. **Se desactiva automáticamente después de 12 horas.** Para auditorías financieras a largo plazo, enrutar a Blob Storage.

1.4 Azure Functions

Planes de hospedaje

Plan	Escalado	Timeout max	VNet	Característica clave
Consumption	Automático (serverless)	5 min (config hasta 10 min)	NO	Pago por uso — escala a cero
Flex Consumption	Dinámico por concurrencia	Sin límite	SÍ	Instancias preaprovisionadas, cold start reducido
Premium	Automático con instancias precalentadas	Sin límite	SÍ	Sin cold start, ejecución larga

Plan	Escalado	Timeout max	VNet	Característica clave
Dedicated (App Service)	Manual/automático según plan	Sin límite	Sí	Reutiliza plan App Service existente
Container Apps	Automático (KEDA)	Sin límite	Sí	Funciones en contenedor con ACA

⚡ CLAVE

Consumption NO soporta VNet integration. Para conectar a recursos en una VNet privada, usar Premium, Flex Consumption o Dedicated.

Premium: instancias pre-calentadas (pre-warmed) eliminan el cold start. El número mínimo es configurable.

Flex Consumption: nuevo plan. Escala por concurrencia configurada por instancia. Permite instancias preaprovisionadas para mitigar cold starts.

🚫 TRAMPA DEL EXAMEN

CRÍTICO — Límite de 230 segundos para HTTP Triggers: Sin importar el plan o timeout configurado, las funciones con **HttpTrigger** tienen un **máximo estricto de 230 segundos** para responder. Esto se debe al timeout del load balancer de Azure.

Solución arquitectónica para reportes financieros pesados (FinanciApp): Si el proceso tarda más de 230 segundos, usar **Durable Functions** o devolver respuesta inmediata + procesar en background.

Consumption timeout: Default = 5 min, máximo configurable en host.json = **10 min**. Flex Consumption, Premium, Dedicated = sin límite.

Funciones vs. otros servicios

Servicio	Modelo	Escalado	Caso de uso
Azure Functions	Código (imperativo)	Automático serverless	Tareas event-driven, procesamiento por lotes, APIs cortas
Logic Apps	Visual (declarativo)	Automático serverless	Integración de sistemas, flujos de trabajo B2B, conectores
WebJobs	Código (imperativo)	Manual o según App Service plan	Background tasks en App Service, legacy, NO autoscale propio

Triggers y bindings

- **Trigger:** define cómo se invoca la función (solo puede haber **1** por función)
- **Input binding:** datos de entrada adicionales (lectura)
- **Output binding:** datos de salida a otros servicios (escritura)
- **Definición de bindings por lenguaje:**
 - **JavaScript/Python:** se definen en **function.json** (editable en portal)

- **C# class library / Java:** se definen con **atributos/anotaciones en el código**. NO dependen de function.json y NO se pueden editar en el portal

Triggers comunes

Trigger	Uso
HttpTrigger	Invocación via HTTP/REST. Authorization levels: anonymous, function, admin
TimerTrigger	Ejecución programada (expresión CRON)
BlobTrigger	Se activa al crear/modificar un blob
QueueTrigger	Mensajes en Azure Storage Queue
ServiceBusTrigger	Mensajes en Service Bus Queue o Topic
EventHubTrigger	Eventos en Azure Event Hubs
CosmosDBTrigger	Cambios en el change feed de Cosmos DB

★ NOTA — HttpTrigger — Niveles de autorización

anonymous: sin API key — apto para APIs públicas

function: requiere function key — nivel por función individual

admin: requiere master key — solo para operaciones administrativas

CRÍTICO: host.json controla configuración GLOBAL de todas las funciones. function.json define los BINDINGS de cada función individual (solo para lenguajes dinámicos).

★ NOTA — Almacenamiento y Archivos de Configuración

Toda Function App **requiere una cuenta de Azure Storage** de propósito general (Blob, Queue, Files, Table) para administrar triggers y registrar ejecuciones.

host.json: opciones globales para todas las funciones (timeout, retry, logging, etc.).

local.settings.json: configuraciones SOLO para desarrollo local. **NUNCA subir a repositorio remoto** — contiene secretos y cadenas de conexión.

TRAMPA de Identidades: Si la Function App corre en Consumption o Premium y necesita conectarse a su propio Azure Files compartido, **NO admite Managed Identity**. Debes usar la cadena de conexión WEBSITE_AZUREFILESConnectionString.

Durable Functions

- Permite escribir flujos de trabajo stateful en Azure Functions
- **Orchestrator function:** coordina la ejecución (no debe realizar I/O directamente)
- **Activity function:** unidad de trabajo que realiza la tarea
- **Entity function:** estado duradero (contador, acumulador) — actores con estado accedidos via EntityId

★ NOTA — Durable Entities — métodos de acceso desde el Orchestrator

SignalEntityAsync: envía una operación a la entidad — fire and forget. No espera respuesta. Usar cuando no necesitas el resultado.

CallEntityAsync: llama a la entidad y espera la respuesta (await). Usar cuando necesitas el valor de retorno.

TRAMPA: usar CallEntityAsync cuando no necesitas respuesta bloquea innecesariamente el Orchestrator.

Patrones clave de Durable Functions

- **Function Chaining:** $A \rightarrow B \rightarrow C$ en secuencia
- **Fan-out/Fan-in:** llama múltiples actividades en paralelo, espera todas
- **Async HTTP APIs:** cliente inicia tarea larga, consulta estado via polling
- **Monitor:** polling periódico hasta condición (reemplaza TimerTrigger para esperas variables)
- **Human interaction:** espera evento externo (aprobación humana)

⚡ CLAVE

Las Orchestrator functions deben ser **DETERMINISTAS**. No uses `DateTime.Now`, `Guid.NewGuid()` ni llames directamente a APIs externos. Usa **`context.CurrentUtcDateTime`** en su lugar.

Cada vez que el Orchestrator se reanuda, replay completo del historial desde el inicio.

🚫 TRAMPA DEL EXAMEN

Output bindings no son transaccionales: si la función falla a mitad, algunos outputs pueden haberse ejecutado y otros no.

Consumption timeout: Default = 5 min, máximo configurable en `host.json` = **10 min**. En Premium/Dedicated/Flex es ilimitado.

🏠 ANALOGÍA BANCARIA

Si FinanciApp necesita generar un reporte de estado de cuenta que tarda 5 minutos: usar **Async HTTP API pattern de Durable Functions**. La API responde inmediatamente con un URL de estado (202 Accepted), el cliente hace polling hasta que el reporte esté listo.

PARTE 2 — Desarrollo para Azure Storage (15-20%)

2.1 Azure Blob Storage

Tipos de cuentas de almacenamiento

Tipo de cuenta	Redundancia	Uso recomendado
General-purpose v2 estándar	LRS, GRS, ZRS, GZRS	Recomendada por defecto. Soporta Blobs, Files, Queues y Tables
Blobs de bloque premium	LRS y ZRS ÚNICAMENTE	Alta transacciones y baja latencia (ej. sistema bancario de transacciones en tiempo real)
Archivos premium	LRS y ZRS	Azure Files para aplicaciones empresariales
Blobs de página premium	LRS	Discos VHD de Azure VMs

Tipos de recursos

- **Cuenta de almacenamiento:** espacio de nombres único. URL base: `http://<cuenta>.blob.core.windows.net`
- **Contenedor:** agrupa blobs. Nombres: 3-63 chars, minúsculas, números y guion, NO puede empezar/terminar con guion, NO guiones consecutivos
- **Blob:** el objeto de datos dentro del contenedor

Tipos de blobs

Tipo	Descripción	Capacidad / Uso típico
Block Blob	Datos binarios o texto. Compuesto de bloques gestionables individualmente	Hasta ~190.7 TiB. Archivos generales, imágenes, videos
Append Blob	Optimizado para operaciones de anexión (append)	Logs, auditorías de transacciones bancarias
Page Blob	Acceso aleatorio a bloques de 512 bytes	Hasta 8 TB. Discos VHD de Azure VMs

Niveles de acceso (Access Tiers)

Nivel	Latencia	Costo almacén.	Costo acceso	Min. días	Cuándo usar
Hot	ms	Alto	Bajo	N/A	Acceso frecuente
Cool	ms	Medio	Medio	30 días	Acceso ocasional
Cold	ms	Bajo	Alto	90 días	Acceso raro
Archive	Horas (rehidratación)	Muy bajo	Muy alto	180 días	Archivado, backups

⚡ CLAVE

Archive está OFFLINE — requiere rehidratación antes de poder acceder. Solo disponible para block blobs individuales, NO a nivel de cuenta.

★ NOTA — Rehidratación desde Archive — 2 métodos y prioridades

Método 1 — Copy Blob: copia el blob archivado a un nuevo blob en nivel Hot/Cool. El blob original **QUEDA en Archive**.

Método 2 — Set Blob Tier: cambia el nivel del mismo blob. Una vez iniciado NO puede cancelarse.

Prioridad Standard: puede tardar hasta **15 horas**.

Prioridad High: menos de **1 hora** para blobs < 10 GB.

Para verificar prioridad de rehidratación en curso: llamar a **Get Blob Properties** y revisar el header **x-ms-rehydrate-priority**.

TRAMPA CRÍTICA: Si usas Set Blob Tier y tienes una lifecycle policy activa, la policy puede re-archivar el blob automáticamente porque la "last modification date" NO cambia durante la rehidratación.

Metadatos y Propiedades

- **Límite de tamaño:** El tamaño total de todos los pares nombre/valor de metadatos **no puede superar los 8 KB**
- **REST API — Prefijo:** Los metadatos personalizados siempre deben llevar el prefijo **x-ms-meta-** (ej. x-ms-meta-documentid: 12345)
- **Sobrescritura total (CUIDADO):** Si haces un PUT para establecer metadatos, sobrescribes **TODOS** los metadatos existentes. Un PUT sin headers de metadatos **borra todos los metadatos** del recurso
- Si envías dos headers de metadatos con el mismo nombre, Blob Storage los **concatena con coma** y devuelve HTTP 200 (no error)

SDK .NET — Jerarquía de clases y código clave

Clase / Namespace	Propósito	Instanciación directa posible
<code>Azure.Storage.Blobs</code>	Namespace raíz — BlobServiceClient, BlobContainerClient, BlobClient	—
<code>Azure.Storage.Blobs.Specialized</code>	CRÍTICO — BlockBlobClient, AppendBlobClient, PageBlobClient	—
<code>BlobServiceClient</code>	Punto de entrada. Representa la cuenta completa. Enumera/crea/elimina contenedores	Sí
<code>BlobContainerClient</code>	Representa un contenedor. Puede obtenerse desde BlobServiceClient o directamente	Sí (sin BlobServiceClient)
<code>BlobClient</code>	Representa un blob individual. Operaciones de lectura/escritura/delete	Sí (con URI + credencial)

📁 SDK / CÓDIGO

```
// Crear BlobServiceClient con Managed Identity (recomendado)
BlobServiceClient svcClient = new(
    new Uri("https://<cuenta>.blob.core.windows.net"),
    new DefaultAzureCredential());

// Obtener BlobContainerClient desde el service client
BlobContainerClient containerClient = svcClient.GetBlobContainerClient("mi-contenedor");

// O directamente sin pasar por BlobServiceClient
BlobContainerClient containerClient = new(
    new Uri("https://<cuenta>.blob.core.windows.net/mi-contenedor"),
    new DefaultAzureCredential());

// Obtener BlobClient
BlobClient blobClient = containerClient.GetBlobClient("archivo.pdf");

// Leer propiedades Y metadatos juntos
BlobProperties props = await blobClient.GetPropertiesAsync();

// Establecer metadatos (PUT sobrescribe TODOS)
IDictionary<string, string> meta = new Dictionary<string, string>();
meta["documentId"] = "12345";
await containerClient.SetMetadataAsync(meta);

// Leer metadatos
BlobContainerProperties p = await containerClient.GetPropertiesAsync();
foreach (var kv in p.Metadata) Console.WriteLine($"{kv.Key}: {kv.Value}");
```

🚫 TRAMPA DEL EXAMEN

`Azure.Storage.Blobs.Specialized` es **CRÍTICO** para el examen. Si necesitas usar `BlockBlobClient`, `AppendBlobClient` o `PageBlobClient`, debes importar este namespace adicional — no vienen en el namespace base.

`GetPropertiesAsync()` recupera propiedades Y metadatos juntos. No hay método separado para solo metadatos.

`BlobContainerClient` puede instanciarse directamente sin necesidad de crear primero un `BlobServiceClient`. Esto aparece frecuentemente en preguntas de completar código.

Lifecycle Management — Políticas

- Definidas en JSON: `rules[]` → `name`, `enabled`, `type: Lifecycle`, `definition` → `filters` + `actions`
- Acciones:** `tierToCool`, `tierToCold`, `tierToArchive`, `delete`, `enableAutoTierToHotFromCool`
- Filtros:** `blobTypes` (requerido), `prefixMatch` (hasta 10 prefijos), `blobIndexMatch` (hasta 10 condiciones de tag)
- Condiciones:** `daysAfterModificationGreaterThan`, `daysAfterCreationGreaterThan`, `daysAfterLastAccessTimeGreaterThan`
- Si múltiples acciones aplican al mismo blob: lifecycle aplica la acción **MÁS ECONÓMICA** (`delete` < `tierToArchive` < `tierToCool`)
- Máximo **100 reglas** por política

- **Actualizaciones:** una política debe leerse o escribirse COMPLETA — no se admiten actualizaciones parciales

📄 SDK / CÓDIGO

```
// Crear lifecycle policy vía CLI
az storage account management-policy create \
--account-name <cuenta> \
--policy @policy.json \
--resource-group <rg>
```

⚡ CLAVE

daysAfterLastTierChangeGreaterThan solo aplica a tierToArchive y define el mínimo de días en tier caliente antes de volver al archivo.

Seguridad — Cifrado

- Todos los datos cifrados con **AES-256** en reposo. Habilitado siempre, no puede desactivarse
- **Claves administradas por Microsoft (default):** Azure gestiona todo
- **Customer-Managed Keys (CMK):** cliente administra claves en Azure Key Vault o Key Vault HSM
- **Customer-Provided Keys (CPK):** cliente incluye la clave en cada request — granular, solo Blob Storage. NO aplica a Queues, Tables ni Files
- **Cifrado del lado del cliente (CSE):** cifrado en la app antes de cargar (v2=GCM recomendado, v1=CBC legacy)

★ NOTA — SAS — Shared Access Signatures

Account SAS: delega acceso a uno o más servicios de Storage (Blob, Queue, Table, File). Firmada con clave de cuenta.

Service SAS: delega acceso a un recurso específico en UN solo servicio. Soporta Stored Access Policies.

User Delegation SAS: SOLO Blob Storage. Firmada con credenciales de Entra ID — **la más segura**. No usa la clave de la cuenta.

Stored Access Policy: permite revocar SAS del lado servidor sin cambiar la URL. Solo disponible en Service SAS, NO en Account SAS.

TRAMPA — Stored Access Policy nueva: puede tardar hasta **30 segundos en surtir efecto**. Sin embargo, revocarla o eliminarla surte efecto de forma **inmediata**.

Para revocar una Account SAS debes rotar o regenerar la clave de la cuenta.

📄 SDK / CÓDIGO

```
// Crear Stored Access Policy con CLI
az storage container policy create \
--account-name <cuenta> \
--container-name <contenedor> \
--name <policy-name> \
--permissions r \
--expiry 2026-12-31
```

🔊 TRAMPA DEL EXAMEN

Nombres de contenedor: 3-63 chars, solo minúsculas, números y guion. NO puede empezar ni terminar con guion. NO guiones consecutivos.

Metadatos con mismo nombre: si envías dos headers con el mismo nombre, Blob Storage los concatena con coma y devuelve HTTP 200 (no error).

Metadatos PUT sobrescribe TODO: un PUT sin headers de metadatos borra todos los metadatos del recurso.

2.2 Azure Cosmos DB

APIs y modelos de datos

API	Modelo de datos	Cuándo usarla
NoSQL (Core SQL)	JSON / documentos	Default — nueva app, sintaxis SQL familiar
MongoDB	BSON / documentos	Migrar apps MongoDB existentes
Cassandra	Tablas / columnar	Migrar apps Cassandra
Gremlin	Grafos	Relaciones complejas entre entidades (ej. red de fraudes bancarios)
Table	Clave-valor (tablas)	Migrar Azure Table Storage

Niveles de consistencia

Nivel (fuerte → débil)	Garantía	Latencia	Disponibilidad
Strong (Fuerte)	Lee siempre la escritura más reciente	Mayor	Menor
Bounded Staleness	Max K versiones o T tiempo de retraso — SLA de latencia garantizado	Alta	Media
Session (DEFAULT)	Consistencia dentro de la sesión del cliente	Baja	Alta
Consistent Prefix	Nunca ve escrituras fuera de orden (pero no garantiza recency)	Baja	Alta
Eventual	Eventual convergencia sin garantías de orden	Menor	Mayor

⚡ CLAVE

Session es el nivel DEFAULT y el más usado. **Strong** NO está disponible en escrituras multi-región.

Bounded Staleness: dos parámetros — K (número de versiones) y T (intervalo de tiempo). Es el único nivel con **SLA de latencia garantizado**.

Consistent Prefix: NUNCA verás escrituras fuera de orden, pero no garantiza que leas lo más reciente.

★ NOTA — Consistent Prefix vs Eventual — diferencia crítica

Consistent Prefix: garantiza ORDEN pero NO garantiza recency (puedes leer datos atrasados, pero siempre en orden correcto)

Eventual: NO garantiza ni orden ni recency. Puedes ver B antes que A, o no ver ninguno todavía.

Regla para el examen: "nunca fuera de orden" → Consistent Prefix. "eventual convergencia sin garantías" → Eventual.

Particionamiento y rendimiento

- **Partition key:** define cómo se distribuyen los datos. Alta cardinalidad = distribución uniforme
- **CRÍTICO:** la partition key es **INMUTABLE** — no puede cambiarse después de crear el contenedor
- **Partición física:** mecanismo de almacenamiento subyacente — hasta **50 GB** de datos y **10,000 RU/s** de rendimiento
- **Partición lógica:** abstracción sobre la física — límite estricto de **20 GB** de datos
- **Request Units (RU/s):** moneda de Cosmos DB. Una lectura de 1KB = 1 RU. Una escritura de 1KB ≈ 5 RU
- **Rendimiento aprovisionado:** escala en incrementos exactos de **100 RU/s**
- **Autoscale:** escala entre 10% y 100% del max RU configurado. Si configuras 1000 RU max, el mínimo será 100 RU (no 0)
- **Serverless:** paga por consumo exacto. NO tiene RU garantizados. Distinto a Autoscale
- **Disponibilidad multi-región:** **99.999%** para operaciones de lectura y escritura con múltiples regiones. Agregar/quitar regiones NO requiere pausar la app
- **Stored procedures, triggers y UDFs:** ejecutan en la misma partición lógica (ACID dentro de partición)
- **Change Feed:** stream de cambios (inserts y updates) — **NO incluye deletes** (usar soft delete para capturarlos)

SDK .NET v3 — Jerarquía completa y patrones clave

Clase	Propósito	Notas críticas
CosmosClient	Punto de entrada. Representa la cuenta. Thread-safe.	Singleton — una sola instancia por app. Permite gestión eficiente de conexiones.
Database	Obtenida desde CosmosClient. Contiene contenedores.	CreateDatabaseAsync lanza excepción si ya existe. Usar CreateDatabaseIfNotExistsAsync para idempotencia.
Container	Obtenida desde Database. Es la colección de ítems.	Proveer siempre new PartitionKey(valor) en operaciones CRUD.
FeedIterator<T>	Resultado de GetItemQueryIterator . Cursor de paginación.	Iterar con while (iterator.HasMoreResults) { var page = await iterator.ReadNextAsync(); }

📄 SDK / CÓDIGO

```
// ① CosmosClient como Singleton
CosmosClient client = new CosmosClient(endpoint, key);
```

```
// ② Crear base de datos (idempotente)
Database db = await client.CreateDatabaseIfNotExistsAsync("FinanciDB");

// ③ Crear contenedor con partition key
Container container = await db.CreateContainerIfNotExistsAsync("Cuentas", "/accountId");

// ④ Insertar ítem — PartitionKey es OBLIGATORIO
ItemResponse<Cuenta> resp = await container.CreateItemAsync(
    cuenta, new PartitionKey(cuenta.AccountId));

// ⑤ Leer ítem — PartitionKey + id
ItemResponse<Cuenta> resp = await container.ReadItemAsync<Cuenta>(
    id, new PartitionKey(accountId));

// ⑥ Consultar con SQL — GetItemQueryIterator devuelve FeedIterator
FeedIterator<Cuenta> iter = container.GetItemQueryIterator<Cuenta>(
    new QueryDefinition("SELECT * FROM c WHERE c.tipo = @tipo")
        .WithParameter("@tipo", "corriente"),
    requestOptions: new QueryRequestOptions {
        PartitionKey = new PartitionKey("Account1") });

while (iter.HasMoreResults)
{
    FeedResponse<Cuenta> page = await iter.ReadNextAsync();
    foreach (var item in page) Console.WriteLine(item.AccountId);
}
```

★ NOTA — Triggers, Stored Procedures y UDFs (JavaScript)

Pre-triggers: NO pueden recibir parámetros de entrada. Usan el objeto de solicitud para manipular el mensaje antes de la operación.

Registro de trigger: debes especificar la operación permitida (ej. TriggerOperation.Create). Un trigger de Create NO puede usarse en una operación de Replace.

Post-triggers: operan dentro de la misma transacción. Si el post-trigger lanza una excepción, **toda la transacción se revierte**.

Arrays en portal: los arrays pasados como parámetros a un stored procedure en el portal se envían como strings. Tu código JS debe hacer **JSON.parse()** para procesarlos.

Los stored procedures pueden implementar un **modelo de continuación** para procesar por lotes sin agotar el tiempo de ejecución.

★ NOTA — Change Feed Processor — 4 componentes

Contenedor supervisado: donde están los datos que se monitorean

Contenedor de concesión (Lease): almacena el estado del procesamiento — coordina múltiples instancias

Instancia de proceso: el host que ejecuta el procesador (VM, App Service, Azure Function)

Delegado: el código que procesa cada lote de cambios

Modelo de datos jerárquico

- **Cuenta → Base de datos → Contenedor → Items**
- **TTL a nivel contenedor:** define el default de expiración en segundos
- **TTL a nivel item:** sobrescribe el default del contenedor. TTL = -1 en item = NUNCA expira, aunque el contenedor tenga TTL configurado
- **Indexing:** por defecto todos los campos se indexan automáticamente. Se puede personalizar

⚡ CLAVE

Stored procedures SOLO dentro de una partición lógica. No pueden cruzar particiones.

Autoscale ≠ Serverless: Autoscale va de 10% a 100% del max. Serverless paga por consumo exacto. Serverless NO tiene RU garantizados.

Partición lógica máx: 20 GB. Partición física máx: 50 GB y 10,000 RU/s.

🚫 TRAMPA DEL EXAMEN

Partition key IMMUTABLE: no puede cambiarse después de crear el contenedor. En FinanciApp, elegir bien la partition key (ej. accountId) desde el inicio es crítico.

Strong consistency NO disponible en multi-región. Si el examen pide consistencia fuerte + multi-región → no es posible con Strong.

PARTE 3 — Implementación de Seguridad de Azure (15-20%)

3.1 Microsoft Entra ID — Autenticación y Autorización

Objeto de Aplicación vs. Entidad de Servicio

- **Objeto de Aplicación (Application Object):** plantilla global (blueprint) que reside SOLO en el **tenant principal (home tenant)**. Relación 1:1 con tu software
- **Entidad de Servicio (Service Principal):** instancia local de la aplicación en un tenant específico
- **App multi-tenant:** 1 Application Object → múltiples Service Principals (uno por cada tenant que use la app)

TRAMPA DEL EXAMEN

Application Object ≠ Service Principal. Application Object es la plantilla global. Service Principal es la instancia local en cada tenant.

Flujos de OAuth 2.0 y OpenID Connect

- **Authorization Code Flow:** apps web con servidor backend. Intercambia código por tokens
- **PKCE:** extensión del auth code flow para SPAs y apps móviles (sin client secret). NO es un flujo separado
- **Client Credentials Flow:** daemon/servicios sin usuario. Solo tokens de app (no delegados)
- **On-Behalf-Of (OBO):** API intermedia llama a otra API en nombre del usuario original
- **Device Code Flow:** dispositivos sin browser (IoT, CLI, smart TVs)
- **Implicit Flow: OBSOLETO** — no usar en apps nuevas. Reemplazado por Auth Code + PKCE
- **ROPC (Resource Owner Password Credentials):** el más inseguro. Solo para sistemas legacy

★ NOTA — Cliente Público vs Cliente Confidencial — Código MSAL.NET

Cliente Público: SPA, Mobile, Desktop. NO puede guardar secretos de forma segura.

Cliente Confidencial: Web App, Web API, Daemon. Puede guardar client secret o certificado.

TRAMPA MSAL: los modificadores `.WithCertificate()` y `.WithClientSecret()` son **mutuamente excluyentes**. Usarlos juntos lanza excepción.

Certificado > Client Secret: certificado es más seguro porque no viaja por la red en el intercambio

SDK / CÓDIGO

// Cliente PÚBLICO (SPA, mobile, desktop)

```
IPublicClientApplication app = PublicClientApplicationBuilder  
    .Create(clientId)  
    .Build();
```

// Cliente CONFIDENCIAL con client secret

```
IClientApplication app = ConfidentialClientApplicationBuilder  
    .Create(clientId)  
    .WithClientSecret(clientSecret)  
    .WithAuthority(new Uri($"https://login.microsoftonline.com/{tenantId}"))  
    .Build();
```

// Cliente CONFIDENCIAL con certificado (más seguro)

```
IClientApplication app = ConfidentialClientApplicationBuilder
```

```
.Create(clientId)
.WithCertificate(certificate) // NO combinar con .WithClientSecret()
.Build();
```

★ NOTA — Consentimiento incremental y de administrador

Consentimiento dinámico/incremental: puedes solicitar permisos adicionales a medida que el usuario usa más features, sin pedir todo al inicio.

TRAMPA del Consentimiento Dinámico: NO funciona bien para permisos que requieren consentimiento de administrador. Si necesitas permisos de alto privilegio, debes registrarlos **estáticamente** en el portal.

Ámbito por defecto en MSAL: si omites el identificador de recurso en el parámetro scope, el sistema asume que estás pidiendo acceso a **Microsoft Graph**.

Conditional Access puede interrumpir flujos silenciosos (OBO). Tu app debe manejar el "claims challenge" y redirigir al usuario a autenticación interactiva

Tokens

- **Access Token:** JWT que autoriza el acceso a recursos. Vida corta (~1 hora)
- **Refresh Token:** permite obtener nuevos access tokens sin login. Vida larga
- **ID Token:** contiene información de identidad del usuario (OpenID Connect)
- **Claims:** pares clave-valor dentro del JWT (oid, tid, roles, scp, etc.)
- **Scopes Delegated:** actúan en nombre del usuario (usuario debe estar logueado)
- **Scopes Application:** actúan como la app (no hay usuario — daemon/servicio)

Registro de aplicaciones

- **Tenant:** "common" o "consumers" para multi-tenant, tenant ID específico para single-tenant
- **Client ID:** identificador público de la app (GUID)
- **Client Secret o Certificate:** credencial de la app (certificate = más seguro)
- **Redirect URI:** URL donde Entra ID envía el código de autorización
- **API Permissions:** Delegated = en nombre del usuario, Application = solo app
- **Admin consent:** permisos de alto privilegio requieren aprobación del admin

⚡ CLAVE

MSAL es la librería recomendada. Maneja caché de tokens, renovación y los flujos OAuth automáticamente.

Azure AD Graph está DEPRECADO — siempre usar Microsoft Graph.

🔒 TRAMPA DEL EXAMEN

PKCE no es un flujo independiente: es una EXTENSIÓN del Authorization Code flow para clientes públicos que no pueden guardar client secret.

Permisos delegados vs application: Delegated necesita usuario activo. Application permissions son para daemons/servicios.

3.2 Managed Identities (Identidades Administradas)

Característica	System-Assigned	User-Assigned
Creación	Como parte del recurso Azure	Recurso Azure independiente (2 pasos: az identity create → az vm identity assign)
Ciclo de vida	Se elimina con el recurso	Ciclo de vida independiente
Compartir entre recursos	No — solo 1 recurso	Sí — múltiples recursos pueden compartirla
Caso de uso	Carga de trabajo en un solo recurso	Múltiples recursos con mismos permisos, o permisos preconfigurados
Roles requeridos asignación	N/A	Colaborador de máquina virtual + Operador de identidad administrada

⚡ CLAVE

Usar MI elimina la necesidad de credenciales en el código. Se recomienda **DefaultAzureCredential** que detecta automáticamente el tipo de credencial.

MI es un tipo de Service Principal administrado por Azure. La diferencia clave: Azure gestiona automáticamente las credenciales.

Comandos CLI completos — Managed Identities

📁 SDK / CÓDIGO

// == SYSTEM-ASSIGNED MI ==

// Crear VM nueva CON system-assigned MI desde el inicio

```
az vm create \
  --resource-group <rg> \
  --name myVM \
  --image Ubuntu2204 \
  --assign-identity \      # activa System-Assigned
  --role contributor \    # rol en la suscripción
  --scope /subscriptions/<id> # ámbito del rol
```

// Asignar system-assigned MI a VM EXISTENTE

```
az vm identity assign \
  --resource-group <rg> \
  --name myVM
```

// == USER-ASSIGNED MI (proceso de 2 pasos) ==

// PASO 1: Crear la identidad como recurso independiente

```
az identity create \
  --resource-group <rg> \
  --name myUserAssignedIdentity
```

```
// PASO 2: Asignar la identidad a la VM
az vm identity assign \
--resource-group <rg> \
--name myVM \
--identities <USER-ASSIGNED-IDENTITY-RESOURCE-ID>

// == PARA APP CONFIGURATION ==
// Asignar system-assigned MI a App Config
az appconfig identity assign \
--name myTestAppConfigStore \
--resource-group <rg>

// Asignar user-assigned MI a App Config
az appconfig identity assign \
--name myTestAppConfigStore \
--resource-group <rg> \
--identities <USER-ASSIGNED-MI-RESOURCE-ID>
```

Flujo de autenticación (System-Assigned)

- Azure Resource Manager habilita la identidad → crea Service Principal en Entra ID
- **IMDS endpoint:** <http://169.254.169.254/metadata/identity/oauth2/token> (solo accesible desde dentro del recurso)
- El código solicita token al IMDS → Entra ID devuelve JWT
- El código usa el JWT para llamar al recurso de Azure destino

DefaultAzureCredential — Orden de autenticación

- 1. Variables de entorno (AZURE_CLIENT_ID, AZURE_CLIENT_SECRET, AZURE_TENANT_ID)
- 2. Identidad administrada (si la app corre en Azure con MI habilitada)
- 3. Visual Studio (credenciales del IDE en desarrollo)
- 4. Azure CLI (az login)
- 5. Azure PowerShell (Connect-AzAccount)
- 6. Explorador interactivo (deshabilitado por defecto)

SDK / CÓDIGO

```
// DefaultAzureCredential — funciona igual en local y en Azure
// using Azure.Identity;
var credential = new DefaultAzureCredential();

// Con User-Assigned MI específica (cuando hay varias)
var credential = new DefaultAzureCredential(
    new DefaultAzureCredentialOptions {
        ManagedIdentityClientId = "<user-assigned-client-id>"
    });

// Usar con BlobClient
var blobClient = new BlobClient(
    new Uri("https://myaccount.blob.core.windows.net/mycontainer/myblob"),
```

```
credential);
```

```
// ChainedTokenCredential — orden personalizado  
var credential = new ChainedTokenCredential(  
    new ManagedIdentityCredential(), // primero MI (producción)  
    new AzureCliCredential());      // fallback CLI (local)
```

🔒 TRAMPA DEL EXAMEN

Si eliminas un recurso con **System-Assigned MI**: la MI se elimina automáticamente. PERO si la MI estaba en un Key Vault access policy, el acceso se rompe de inmediato.

User-Assigned MI es un proceso de 2 pasos: primero crear la identidad (az identity create), luego asignarla (az vm identity assign). El examen puede pedir el paso en secuencia.

MI no soporta todos los servicios: solo servicios que soportan autenticación de Entra ID.

3.3 Azure Key Vault

Tipos de objetos

- **Secrets**: cadenas de texto plano (connection strings, passwords, API keys). Sí exportables
- **Keys**: claves criptográficas (RSA, EC). Operaciones: encrypt, decrypt, sign, verify, wrapKey, unwrapKey. **NO exportables en Premium/HSM**
- **Certificates**: certificados X.509. Incluye la clave privada asociada. Auto-renovación configurable

⚡ CLAVE

No guardes claves privadas como Secrets: usa el tipo **Keys** (con HSM en Premium) para que no sean exportables. Secrets son texto plano exportable.

Niveles de servicio

- **Standard**: secretos, claves software-protected
- **Premium**: agrega soporte para Hardware Security Modules (HSM) — claves no exportables
- **Managed HSM**: HSM dedicado para control completo de claves

Acceso y autenticación

- **Modelo de acceso**: plano de gestión (admin del vault) + plano de datos (acceso a secretos/claves/certs)
- **Access Policies (modelo heredado)**: permisos a nivel de vault para identidades
- **RBAC (modelo recomendado)**: roles estándar de Azure (Key Vault Secrets Officer, Key Vault Reader, etc.)
- **Soft Delete**: retiene objetos eliminados por 7-90 días (configurable). Habilitado por defecto
- **Purge Protection**: impide eliminar definitivamente objetos durante el período de retención. Requerido para CMK con Cosmos DB/Storage. **UNA VEZ HABILITADA NO PUEDE DESACTIVARSE**

★ NOTA — Cifrado en tránsito y REST API

Key Vault usa TLS obligatoriamente con **Perfect Forward Secrecy (PFS)** y claves RSA de **2048 bits**.

Si llamas a la API REST de Key Vault sin token (o con uno inválido): devuelve **HTTP 401 Not Authorized** con el header **WWW-Authenticate** que contiene la URI del servicio de autorización.

⚡ CLAVE

Para acceder desde código: usar DefaultAzureCredential + SecretClient/KeyClient. URI del vault: `https://<nombre>.vault.azure.net/`

Purge Protection es IRREVERSIBLE. Una vez habilitada no puede deshabilitarse.

Best Practices

- Una vault por aplicación por entorno (dev/staging/prod) para aislamiento
- Habilitar diagnostics logging para auditar accesos
- Rotación de secretos automática con Event Grid + Azure Functions
- Usar referencias a Key Vault en App Service/Functions para no hardcodear URIs

★ NOTA — Key Vault Firewall y Private Endpoint

Firewall de Key Vault: permite restringir acceso solo a IPs específicas o VNets

Private Endpoint: asigna una IP privada dentro de tu VNet al vault — el tráfico nunca sale a internet público. Opción más segura.

TRAMPA: si habilitas el firewall y olvidas agregar la IP de tu App Service o la VNet, la app pierde acceso al vault — error **403 Forbidden** en runtime.

3.4 Azure App Configuration

- Almacén centralizado para configuración de aplicaciones y feature flags
- Complementa Key Vault: App Config para configuración no sensible, Key Vault para secretos
- **Etiquetas (labels):** misma clave con diferentes valores por entorno (dev/staging/prod)
- **Snapshot point-in-time:** reproducir configuración en un momento dado — permite rollback de config
- **Sentinel key pattern:** cambio dinámico de configuración sin reiniciar la app
- **Feature Management:** activar/desactivar funcionalidades sin redespargar código
- **Límite de clave-valor:** máximo **10,000 caracteres** combinados por cada par clave-valor (nombre + valor + atributos)
- **Caracteres reservados en claves:** *, comma (,) y backslash (\). Si los necesitas, escápalos con \

⚡ CLAVE

Las referencias a Key Vault en App Configuration **NO almacenan el secreto** — solo almacenan la URI del secreto en Key Vault. La app necesita permisos propios para leer el KV en tiempo de ejecución.

★ NOTA — Feature Flags — Evaluación de Filtros Múltiples

Si un Feature Flag tiene múltiples filtros (ej. filtro por porcentaje de usuarios + filtro por geografía):

Los filtros se evalúan **en orden**. En el momento en que **uno** devuelve True, la característica se enciende y **se omiten los demás filtros**.

Es un OR short-circuit, no un AND.

★ NOTA — CMK en App Configuration

Para cifrar valores con clave propia (CMK) en Key Vault, la Managed Identity de App Configuration

necesita permisos: **GET**, **WRAP** y **UNWRAP**.

Key Vault debe tener activado **Soft-Delete** y **Purge-Protection** para CMK.

PARTE 4 — Supervisión, Solución de Problemas y Optimización (5-10%)

4.1 Azure Monitor y Application Insights

Componentes de Azure Monitor

- **Métricas:** valores numéricos en el tiempo (CPU%, requests/sec, latencia). Retención: 93 días
- **Logs (Log Analytics):** datos estructurados/semi-estructurados con **Kusto Query Language (KQL)**
- **Alertas:** reglas que disparan acciones (email, webhook, runbook) cuando una condición se cumple
- **Workbooks:** dashboards interactivos que combinan métricas, logs y textos
- **Insights:** experiencias pre-construidas (Application Insights, Container Insights, etc.)

Application Insights

- SDK-based (instrumentación de código) o auto-instrumentación (sin código para App Service, Functions, etc.)
- **Telemetría:** requests, dependencies, exceptions, traces, custom events, custom metrics
- **Correlation:** trace distribuido mediante W3C Trace Context (operationId, parentId)
- **Mapa de aplicación:** visualización de dependencias entre componentes
- **Availability tests:** ping tests y multi-step web tests desde ubicaciones globales
- **Smart Detection:** detección automática de anomalías en rendimiento y fallos
- **Live Metrics Stream:** visualización en tiempo real sin latencia de ingestión

★ NOTA — Métricas Estándar vs. Métricas basadas en Registros

Métricas Pre-agregadas (Standard Metrics): el punto de conexión de recopilación preagrega los eventos ANTES del muestreo de ingesta. Por tanto, el muestreo de ingesta **NUNCA afecta su precisión**.

Métricas basadas en Registros: Sí reducen su precisión al verse afectadas por el muestreo y el filtrado.

GetMetric(): método preferido para enviar métricas personalizadas. Preagrega los datos localmente en el SDK, reduciendo ingesta de datos y costo.

★ NOTA — Tipos de Sampling en Application Insights

Adaptive (default en SDK): ajusta automáticamente el porcentaje según el volumen de telemetría

Fixed rate: tú defines el porcentaje fijo de telemetría a capturar

Ingestion sampling: ocurre en el portal/backend, no en el SDK. No ahorra costos de red

TRAMPA: el sampling puede hacer que NO veas TODOS los errores. Para capturar el 100% de excepciones, configura sampling = 100% o excluye exceptions del sampling.

Ingestion sampling NO afecta las métricas pre-agregadas. Solo afecta métricas basadas en registros.

★ NOTA — Application Map

¿Cómo sabe el Mapa de Aplicación diferenciar un microservicio de otro si comparten el mismo recurso de App Insights?

Utiliza la propiedad **cloud_RoleName**. Debes establecerla manualmente en el código de cada componente para que aparezcan como nodos separados en el mapa.

Si todos tienen el mismo cloud_RoleName → aparecen como un solo nodo.

★ NOTA — Pruebas de Disponibilidad

URL Ping Test: se retirará el 30 de septiembre de 2026.

Standard Test: reemplazo oficial. Valida respuesta + certificado TLS/SSL + verbos HTTP (GET, HEAD, POST) + encabezados personalizados.

Prueba personalizada: usar el método **TrackAvailability()** en el código para enviar resultados propios a Application Insights.

★ NOTA — Equivalencias OpenTelemetry

Microsoft está alineando App Insights con OpenTelemetry. El examen puede usar la nueva nomenclatura:

Operation ID → **Trace ID**

ID / Operation Parent ID → **Span ID**

Requests → **Server Spans**

Dependencies → **Other Span Types** (Client, Internal)

Traces → **Logs**

✂ CLAVE

Connection String vs Instrumentation Key: usar siempre Connection String. La Instrumentation Key sola está en desuso.

Para tracing distribuido entre microservicios: AMBOS servicios deben estar instrumentados y usar W3C Trace Context.

Instrumentación manual obligatoria si necesitas registrar **eventos y métricas personalizados** o control estricto del flujo de telemetría.

KQL — Consultas clave

- requests | where resultCode == '500' | summarize count() by bin(timestamp, 1h)
- dependencies | where duration > 1000 | order by duration desc
- exceptions | summarize count() by type, outerMessage
- customEvents | where name == 'Purchase' | extend props = customDimensions | project timestamp, props

Log Analytics Workspace

- Centraliza datos de Application Insights, Azure Monitor, Security Center, etc.
- **Retención:** 30 días default gratuito — configurable hasta 730 días (pago). Importante para compliance/auditoría bancaria
- **Workspace-based AppInsights:** recomendado. Classic AppInsights está deprecado

PARTE 5 — Conexión y Consumo de Servicios (20-25%)

5.1 Azure API Management (APIM)

Componentes principales

- **API Gateway:** punto de entrada para todas las llamadas. Aplica políticas
- **Developer Portal:** portal auto-generado para documentación y pruebas de APIs
- **Management Plane:** interfaz de administración (Azure Portal, ARM, REST API)
- **Backend:** servicio al que APIM proxee las solicitudes
- **Productos:** agrupan una o más APIs con políticas de acceso y cuotas. Las subscriptions son a PRODUCTOS, no directamente a APIs

★ NOTA — Grupos de Usuarios APIM — Inmutables

APIM tiene **3 grupos del sistema** que no se pueden borrar ni modificar:

Administradores: Gestionan la instancia (administradores de la suscripción de Azure).

Desarrolladores: Usuarios autenticados en el portal de desarrolladores.

Invitados: Usuarios NO autenticados. Solo pueden ver APIs (lectura), pero no llamarlas.

★ NOTA — Gateways: Managed vs Self-hosted

Managed: gateway por defecto en Azure.

Self-hosted: es un **contenedor** opcional. Útil en escenarios híbridos/multi-nube donde quieres ejecutar el gateway FUERA de Azure pero administrarlo desde la instancia APIM en Azure.

Políticas (Policies) — Orden de ejecución

- **inbound:** autenticación, rate limiting, transformar request, cache check
- **backend:** modificar URL de backend, set-backend-service
- **outbound:** transformar response, agregar headers, aplicar cache
- **on-error:** manejar errores, devolver respuestas personalizadas

★ NOTA — Elemento <base /> — CRÍTICO

Sin **<base />** en la sección inbound de una política de operación: las políticas de nivel API y Global **NO se ejecutan**. Esto puede hacer que validate-jwt se salte para esa operación.

Los 4 niveles de scope de políticas: Global (todas las APIs) → Workspace → API → Operación
cache-lookup debe estar en INBOUND. **cache-store** debe estar en OUTBOUND.

Políticas comunes

- **rate-limit-by-key:** limita llamadas por clave (IP, subscription, etc.)
- **quota-by-key:** cuota total de llamadas en un período
- **cache-lookup / cache-store:** cachear respuestas (lookup en inbound, store en outbound)
- **validate-jwt:** validar y verificar tokens JWT (va en INBOUND, antes del backend)
- **set-header:** agregar/modificar headers
- **rewrite-uri:** transformar la URL antes de enviar al backend
- **mock-response:** devuelve respuesta mock sin llamar al backend. Prioridad: 1) Ejemplos del esquema, 2) Generado desde esquema, 3) Respuesta vacía
- **forward-request:** enviar la solicitud al backend (política final en backend)

- **choose:** funciona como switch/if-else. Evalúa bloques <when> en orden. Si ninguno es verdadero, ejecuta <otherwise>
- **retry:** ejecuta políticas secundarias y reintenta si falla, basado en condition y count (intentos máximos)

⚡ CLAVE

Las políticas se definen en XML y pueden usar expresiones de política (C# simplificado) encapsuladas en @(expresion).

validate-jwt va en inbound: valida tokens ANTES de que la solicitud llegue al backend.

Niveles de servicio APIM

Tier	SLA	Características clave
Consumption	0%	Serverless, pago por llamada, limitado en políticas. NO usar en producción crítica. NO soporta VNet
Developer	N/A	Solo para desarrollo/pruebas, NO para producción, tiene portal developer
Basic	SÍ	Producción de baja escala, sin VNET
Standard	SÍ	Producción, VPN externa
Premium	SÍ	Multi-región, VNET interna/externa, alta disponibilidad, zona de disponibilidad

★ NOTA — Subscription Keys en APIM

Header: **Ocp-Apim-Subscription-Key** (o como query param api-key)

Cada suscripción tiene Primary y Secondary key. Para rotación sin downtime: (1) cambiar clientes a Secondary, (2) regenerar Primary, (3) volver clientes a Primary

🚫 TRAMPA DEL EXAMEN

<base /> ausente: si una política de operación no tiene <base /> en inbound, las políticas de nivel API y Global no se ejecutan para esa operación.

Consumption tier: 0% SLA. No soporta VNet integration. No usar en producción para APIs críticas (ej. APIs de core bancario).

5.2 Azure Event Grid

Modelo de eventos

- **Publisher:** quien genera el evento (Azure Storage, IoT Hub, custom source)
- **Topic:** endpoint donde los publishers envían eventos. System topics (recursos Azure), Custom topics, Partner topics (SaaS de terceros: Auth0, Salesforce)
- **Subscription:** configuración que define qué eventos van a qué handler, con filtros
- **Handler:** quien recibe y procesa el evento (Azure Function, Webhook, Service Bus, Event Hubs, etc.)

⚡ CLAVE

Event Grid sigue el modelo **push**. Garantiza entrega al menos una vez — puede entregar el mismo evento **MÁS DE UNA VEZ**. Tu handler debe ser **IDEMPOTENTE**.

Orden de eventos NO garantizado. Si necesitas orden, usar Event Hubs (por partición) o Service Bus (con sesiones).

Soporta Event Grid schema Y CloudEvents 1.0 schema (estándar CNCF).

Estructura del evento

- **id**: identificador único del evento
- **source**: recurso que generó el evento
- **type**: tipo de evento (ej: Microsoft.Storage.BlobCreated)
- **data**: payload del evento específico del tipo
- **time**: timestamp del evento
- **dataVersion**: versión del esquema del payload
- **Tamaño máximo de evento: 1 MB**. Eventos > 64 KB se facturan en incrementos de 64 KB (ej. 130 KB = 3 unidades de facturación)

★ NOTA — Validación de Webhooks (TRAMPA CLÁSICA)

Al usar un Webhook propio (ej. Azure Function genérica), debes probar que eres el dueño del endpoint:

Síncrono: Event Grid envía un evento con **validationCode**. Tu app debe devolver ese código inmediatamente en la respuesta HTTP 200.

Asíncrono (Validación Manual): Event Grid envía un evento con una **validationUrl**. Tienes **5 minutos** para hacer GET a esa URL. Durante ese tiempo, la suscripción estará en estado **AwaitingManualAction**.

Filtrado de suscripciones

- **Por tipo de evento**: `includedEventTypes: ["Microsoft.Storage.BlobCreated"]`
- **Por asunto**: `subjectBeginsWith`, `subjectEndsWith`
- **Filtrado avanzado**: operadores sobre campos del data payload (`NumberGreaterThan`, `StringContains`, etc.)

Patrones de uso y DLQ

- **Dead Letter**: eventos no entregados van a un Storage Account configurado. Por defecto **NO está activada** — debes configurarla al crear la suscripción
- **HTTP 400 o HTTP 413 recibidos del handler**: Event Grid **NO reintenta**. Envía directamente a DLQ o descarta
- **Retry**: política de reintento exponencial hasta **24 horas**

🚫 TRAMPA DEL EXAMEN

DLQ no está activada por defecto. Debes configurar un contenedor de Storage al crear la suscripción.

HTTP 400 o 413: Event Grid NO reintenta. Va directo a DLQ o se descarta.

5.3 Azure Event Hubs

Conceptos fundamentales

- Servicio de ingesta de datos masivos (Big Data streaming): millones de eventos/segundo
- **Compatible con Apache Kafka** (sin cambios de código en clientes Kafka)
- Protocolo **AMQP 1.0** como protocolo nativo
- Event Hub = registro distribuido de solo anexión, dividido en particiones

Componentes clave

- **Namespace:** contenedor de administración para uno o más Event Hubs
- **Event Hub:** canal de transmisión (equivalente a un topic de Kafka)
- **Partición:** segmento independiente del log. El número se define al crear el Hub y **NO puede cambiarse después**
- **Consumer Group:** vista lógica del Event Hub para un conjunto de consumidores. 1 Consumer Group por aplicación consumidora
- **Offset:** posición de un consumidor dentro de una partición
- **Checkpoint:** marca el último offset procesado exitosamente

★ NOTA — Retention Period — dato crítico

Standard tier: 1-7 días (default 1 día).

Premium/Dedicated: hasta 90 días.

CRÍTICO: si el consumidor tarda más que el retention period, los eventos se pierden permanentemente.

Unidades de rendimiento

- **Throughput Unit (TU) — Plan Estándar:** 1MB/seg o 1000 eventos/seg entrada, 2MB/seg o 2000 eventos/seg de salida
- **Processing Unit (PU) — Plan Premium:** mayor rendimiento
- **Event Hubs Capture:** captura automática a Blob Storage o Data Lake en formato **Apache Avro**

★ NOTA — Event Hubs Capture — Estructura de archivos

TRAMPA: Capture guarda archivos en formato de carpetas basadas en fecha. Estructura exacta:

{Namespace}/{EventHub}/{PartitionId}/{Year}/{Month}/{Day}/{Hour}/{Minute}/{Second}

Política "primer ganador": la captura se dispara cuando primero se cumpla: el tiempo configurado O el tamaño configurado.

Si no hay datos, Capture escribe **archivos vacíos** para mantener una cadencia predecible.

★ NOTA — Consumer Groups — Mejores Prácticas

Si necesitas leer eventos para dos propósitos distintos (ej. guardar en Storage Y hacer análisis), debes crear **dos Consumer Groups distintos**, uno por cada aplicación.

Un Consumer Group = una vista independiente del stream.

Procesamiento con EventProcessorClient

- Recomendado para producción (vs EventHubConsumerClient para escenarios simples)
- Distribución automática de particiones entre instancias del procesador
- **Checkpoint Store:** necesario para balanceo y recuperación. Típicamente usa Azure Blob Storage
- Patrón consumidor particionado: cada partición procesada por un solo consumidor en un momento dado

SDK / CÓDIGO

```
// EventProcessorClient requiere BlobContainerClient para checkpoint store
var storageClient = new BlobContainerClient(storageConnectionString, blobContainerName);
var processor = new EventProcessorClient(
    storageClient,
    EventHubConsumerClient.DefaultConsumerGroupName,
    ehConnectionString,
    eventHubName);

processor.ProcessEventAsync += async (args) => {
    // Procesar evento
    await args.UpdateCheckpointAsync(); // hacer checkpoint
};
processor.ProcessErrorAsync += async (args) => { /* manejar error */ };
await processor.StartProcessingAsync();
```

✂ CLAVE

El número de particiones define el **máximo de consumidores paralelos** por consumer group. No puedes tener más consumidores activos que particiones.

Orden garantizado SOLO dentro de una partición. Entre particiones, no hay garantía de orden.

TRAMPA DEL EXAMEN

Particiones inmutables: el número de particiones se define al crear el Event Hub y NO puede cambiarse.

Endpoint Kafka vs AMQP: la compatibilidad con Kafka usa un endpoint diferente al AMQP. No son intercambiables en la URL de conexión.

Retention period: si el consumidor tarda más que el período de retención, los eventos se pierden permanentemente.

5.4 Azure Service Bus

Tier	Topics	Capacidades adicionales
Basic	NO	SOLO Queues. Sin Topics ni Subscriptions
Standard	SÍ	Queue + Topics + Subscriptions. Duplicate detection. Capacidad compartida
Premium	SÍ	Todo de Standard + mayor throughput, VNet integration, geo-disaster recovery, zona de disponibilidad. Capacidad dedicada

✂ CLAVE

Topics NO disponibles en tier BASIC. Si el examen pregunta qué tier usar para Topics/Subscriptions → Standard o Premium.

Entidades de mensajería

- **Queue:** mensajería FIFO punto a punto. Un solo consumidor por mensaje
- **Topic + Subscriptions:** mensajería pub/sub. Múltiples suscriptores reciben el mismo mensaje
- **Reglas de suscripción:** filtros SQL o correlación para seleccionar qué mensajes recibe cada suscripción

Características clave

- **At-least-once delivery (PeekLock):** mensaje se bloquea al leer, se confirma (Complete) o abandona (Abandon)
- **Sesiones:** entrega ordenada de mensajes relacionados por session ID
- **Dead-Letter Queue (DLQ):** mensajes que no pueden procesarse. Sub-cola **interna y nativa** de Service Bus. Max delivery count default = **10 intentos**
- **Deferred messages:** procesamiento postergado con SequenceNumber
- **Scheduled messages:** entrega en fecha/hora futura
- **Duplicate detection:** detecta y descarta mensajes duplicados por MessageId. Ventana default = **10 min.** Solo Standard y Premium
- **TTL:** expiración de mensajes si no son procesados
- **Auto-eliminación por inactividad:** configurable — tiempo mínimo **5 minutos**

Proceso de 2 pasos (PeekLock)

- 1. **ReceiveMessages:** obtiene el mensaje y lo hace invisible para otros consumidores (lock)
- 2a. **Complete/DeleteMessage:** confirma el procesamiento exitoso, elimina el mensaje
- 2b. **Abandon:** devuelve el mensaje a la cola para ser reintentado

⚡ CLAVE

Si el código falla entre paso 1 y 2, el lock expira y otro consumidor puede procesar el mensaje. Garantiza que ningún mensaje se pierda.

Message lock renewal: si el procesamiento tarda más que el lock timeout, debes renovar el lock con **RenewMessageLockAsync()** para evitar que otro consumidor tome el mensaje.

★ NOTA — Propiedades de Enrutamiento

Service Bus usa propiedades del sistema para patrones avanzados: **To, ReplyTo, SessionId, CorrelationId.**

Para enrutamiento interno de aplicaciones, usa **propiedades de usuario personalizadas** en lugar de depender de la propiedad reservada "To".

🚫 TRAMPA DEL EXAMEN

ReceiveAndDelete vs PeekLock: ReceiveAndDelete elimina el mensaje en el mismo paso de lectura. Si el código falla, el mensaje SE PIERDE. PeekLock es más seguro para sistemas financieros críticos.

DLQ en Service Bus es una sub-cola interna. En Event Grid la DLQ es un Storage Container externo — son conceptos distintos.

Topics en Basic tier: no disponibles. Solo Queues.

Comparación Service Bus vs Event Grid vs Event Hubs

Servicio	Patrón	Uso típico	Orden	DLQ
Service Bus	Mensajería empresarial (Queue/Topic)	Workflows, transacciones, desacople con garantías. Ej: procesamiento de pagos bancarios	FIFO con sesiones	Sub-cola interna nativa
Event Grid	Eventos reactivos (push)	Reaccionar a cambios de estado en Azure o custom. Ej: nueva transacción genera alerta	No garantizado	Storage Container externo (no activo por defecto)
Event Hubs	Streaming de Big Data	Telemetría, logs, analytics en tiempo real. Ej: stream de movimientos bancarios para detección de fraudes	Por partición	N/A — retention period

5.5 Azure Storage Queues

- Parte de Azure Storage Account — no requiere servicio separado
- **Tamaño máximo de mensaje:** 64 KB
- **Capacidad:** ilimitada — usar cuando necesites almacenar **más de 80 GB** de mensajes (Service Bus no es ideal para eso)
- **Número de mensajes:** ilimitado (hasta el límite de la cuenta de almacenamiento)
- **Visibilidad default al leer:** 30 segundos — configurable
- **Proceso de 2 pasos:** ReceiveMessages (invisible) → DeleteMessage (eliminar)
- **UpdateMessage:** actualiza contenido + extiende visibilidad (útil para checkpointing en procesos largos)
- **PeekMessages:** lee sin hacer invisible (no consume el mensaje)
- **GetProperties** devuelve ApproximateMessagesCount — puede ser MAYOR al número real. No usar para lógica crítica

🚫 TRAMPA DEL EXAMEN

UpdateMessage en Storage Queues permite modificar el CONTENIDO del mensaje mientras está en la cola. En Service Bus NO puedes modificar el contenido de un mensaje en cola.

🔑 CLAVE

Storage Queue vs Service Bus: Storage Queue es más simple y económica (mensajes < 64KB, sin orden garantizado, sin DLQ nativa). Service Bus: mensajes más grandes, DLQ, sesiones, duplicate detection, scheduled messages, transacciones.

Más de 80 GB de mensajes: usar Storage Queue, no Service Bus.

5.6 Microsoft Graph API

- **URL base:** <https://graph.microsoft.com/{version}/{recurso}>
- **Versiones:** v1.0 (producción) y beta (preview — **PROHIBIDO** en producción — puede tener breaking changes sin aviso)
- **Autenticación:** OAuth 2.0 con tokens de Entra ID (MSAL recomendado)
- **Permisos Delegated:** en nombre del usuario. **Application:** solo app, sin usuario

- **Azure AD Graph está DEPRECADO:** siempre usar Microsoft Graph
- **Paginación:** si la respuesta está paginada, usa la propiedad **@odata.nextLink** para obtener la siguiente página
- **Enumeraciones Evolutivas:** Graph puede agregar nuevos valores a enumeraciones sin previo aviso. Tu código debe manejar miembros "desconocidos" sin romperse

Operaciones CRUD

Método HTTP	Operación
GET	Leer datos de un recurso — sin body
POST	Crear recurso o ejecutar acción — requiere body JSON
PATCH	Actualizar recurso con nuevos valores (parcial) — requiere body JSON
PUT	Reemplazar recurso completo — requiere body JSON
DELETE	Eliminar recurso — sin body

SDK de Microsoft Graph (.NET)

- **Microsoft.Graph:** modelos y builders para v1.0
- **Microsoft.Graph.Beta:** para el endpoint beta
- **GraphServiceClient:** acepta `IAAuthenticationProvider`. Recomendado: `DeviceCodeCredential` (apps consola) o `DefaultAzureCredential`
- **Paginación:** usar `PagelIterator` para iterar todas las páginas
- **Batch requests:** hasta **20 requests** individuales en una sola llamada HTTP

⚡ CLAVE

Throttling en Graph: puede devolver HTTP 429 (Too Many Requests). Implementar retry con exponential backoff usando el header **Retry-After**.

REFERENCIA RÁPIDA — Datos Numéricos Clave

Los números exactos suelen aparecer en preguntas del examen:

Servicio / Concepto	Dato / Límite
Blob Storage — Block Blob max size	~190.7 TiB
Blob Storage — Page Blob max size	8 TB
Blob Storage — metadatos max tamaño total	8 KB
Blob Storage — Cool tier min retention	30 días
Blob Storage — Cold tier min retention	90 días
Blob Storage — Archive tier min retention	180 días
Archive — rehidratación Standard	hasta 15 horas
Archive — rehidratación High priority	menos de 1 hora (blobs < 10 GB)
Lifecycle policy — max reglas por política	100
Container name — longitud	3-63 caracteres
Cosmos DB — partición lógica max tamaño	20 GB
Cosmos DB — partición física max tamaño	50 GB
Cosmos DB — partición física max RU/s	10,000 RU/s
Cosmos DB — rendimiento en incrementos de	100 RU/s
Cosmos DB — disponibilidad multi-región	99.999%
Event Hubs — TU estándar entrada	1 MB/seg o 1000 eventos/seg
Event Hubs — TU estándar salida	2 MB/seg o 2000 eventos/seg
Event Hubs — retention Standard	1-7 días (default 1 día)
Event Hubs — retention Premium/Dedicated	hasta 90 días
Event Grid — reintento max tiempo	24 horas
Event Grid — tamaño max evento	1 MB
Event Grid — validación webhook asíncrona	5 minutos para completar
Service Bus — DLQ max delivery count default	10 intentos
Service Bus — duplicate detection window default	10 minutos
Service Bus — auto-eliminación por inactividad mínimo	5 minutos
Storage Queue — tamaño max mensaje	64 KB

Servicio / Concepto	Dato / Límite
Storage Queue — visibilidad default al leer	30 segundos
Storage Queue — usar cuando mensajes >	80 GB total
App Service Autoscale — período recuperación min	5 minutos
App Service Autoscale — duración análisis min	5 minutos
App Service — Application Logging auto-desactiva	después de 12 horas
Functions Consumption — timeout default	5 minutos (max configurable 10 min)
Functions HTTP Trigger — límite estricto load balancer	230 segundos
Key Vault Soft Delete — retención min	7 días
Key Vault Soft Delete — retención max	90 días
Key Vault — TLS cifrado	PFS + claves RSA 2048 bits
App Configuration — max caracteres por clave-valor	10,000 caracteres
Dapr sidecar — puertos	HTTP 3500 / gRPC 50001
Azure AD Access Token — vida típica	~1 hora
Microsoft Graph — max requests en batch	20 requests
Microsoft Graph — versión producción	v1.0
APIM — políticas orden ejecución	inbound → backend → outbound → on-error
ACR — niveles de servicio	Basic, Standard, Premium
Cosmos DB — consistencia default	Session
Cosmos DB — lectura 1KB en RU	1 RU
Cosmos DB — escritura 1KB en RU (aprox)	~5 RU
Cosmos DB — autoscale mínimo RU	10% del max configurado
Certificado privado App Service — clave mínima	2048 bits, cifrado Triple DES, formato PFX
URL Ping Test retiro	30 de septiembre de 2026
Blob SDK namespace especializado	Azure.Storage.Blobs.Specialized (BlockBlobClient, AppendBlobClient, PageBlobClient)
CosmosClient patrón de instanciación	Singleton — una instancia por ciclo de vida de la app

Servicio / Concepto	Dato / Límite
Stored Access Policy — efecto al activar	hasta 30 segundos
Stored Access Policy — efecto al revocar	inmediato

CHEATSHEET — SDK .NET y CLI Críticos

Esta sección concentra las clases, métodos y comandos CLI que el examen evalúa con preguntas de completar código o elegir el comando correcto.

SDK .NET — Mapa de Jerarquías

Servicio	Namespace / Paquete NuGet	Clase principal	Patrón de instanciación
Blob Storage (general)	<code>Azure.Storage.Blobs</code>	<code>BlobServiceClient</code> → <code>BlobContainerClient</code> → <code>BlobClient</code>	Puede instanciarse en cualquier nivel directamente
Blob Storage (especializado)	<code>Azure.Storage.Blobs.Specialized</code>	<code>BlockBlobClient</code> , <code>AppendBlobClient</code> , <code>PageBlobClient</code>	Necesario import adicional — el examen lo evalúa
Cosmos DB	<code>Microsoft.Azure.Cosmos</code>	<code>CosmosClient</code> → <code>Database</code> → <code>Container</code>	Singleton — una sola instancia por app
Key Vault Secrets	<code>Azure.Security.KeyVault.Secrets</code>	<code>SecretClient</code>	<code>new SecretClient(uri, new DefaultAzureCredential())</code>
Key Vault Keys	<code>Azure.Security.KeyVault.Keys</code>	<code>KeyClient</code>	<code>new KeyClient(uri, new DefaultAzureCredential())</code>
Autenticación (todas)	<code>Azure.Identity</code>	<code>DefaultAzureCredential</code> , <code>ChainedTokenCredential</code> , <code>ManagedIdentityCredential</code>	Recomendado: <code>DefaultAzureCredential</code>
MSAL .NET (Entra ID)	<code>Microsoft.Identity.Client</code>	<code>PublicClientApplicationBuilder</code> , <code>ConfidentialClientApplicationBuilder</code>	Público: sin secreto. Confidencial: con secreto o certificado
Event Hubs (consumidor)	<code>Azure.Messaging.EventHubs</code>	<code>EventProcessorClient</code>	Requiere <code>BlobContainerClient</code> para checkpoint store
Service Bus	<code>Azure.Messaging.ServiceBus</code>	<code>ServiceBusClient</code> → <code>ServiceBusSender</code> / <code>ServiceBusReceiver</code>	PeekLock por defecto — confirmar con <code>CompleteMessageAsync</code>
Microsoft Graph	<code>Microsoft.Graph</code>	<code>GraphServiceClient</code>	Acepta <code>DeviceCodeCredential</code> o <code>DefaultAzureCredential</code>

CLI — Comandos críticos por servicio

ACR

📄 SDK / CÓDIGO

```
az acr build --registry <acr> --image <img>:<tag> . # Quick Task (Linux/amd64 por defecto)
az acr build --registry <acr> --image <img>:<tag> --platform Linux/arm64/v8 . # ARM
az acr task create --registry <acr> --name <task> \
--image <img>:<tag> --context <github-url> --git-access-token <tok> # tarea por push
```

ACI

SDK / CÓDIGO

```
az container create --resource-group <rg> --name <n> --image <img> \
--restart-policy OnFailure # política de reinicio
az container create ... \
--secure-environment-variables DB_PASS=valor # variable segura (secureValue)
az container create ... \
--azure-file-volume-account-name <cuenta> \
--azure-file-volume-account-key <clave> \
--azure-file-volume-share-name <share> \
--azure-file-volume-mount-path /mnt/datos # montar Azure Files
```

ACA

SDK / CÓDIGO

```
az containerapp create --name <n> --resource-group <rg> \
--environment <env> --image <img> \
--target-port 80 --ingress external # ingress público
az containerapp update --name <n> --resource-group <rg> \
--image <nueva-img> # actualizar imagen
az containerapp revision list --name <n> --resource-group <rg> # listar revisiones
```

App Service

SDK / CÓDIGO

```
az webapp up --name <n> --resource-group <rg> --runtime "DOTNET|8.0" # deploy directo
az webapp config appsettings set --resource-group <rg> --name <app> \
--settings key=value # configurar settings
az webapp log tail --name <app> --resource-group <rg> # logs en vivo
az webapp list-runtimes --os-type linux # ver runtimes disponibles
```

Managed Identities

SDK / CÓDIGO

```
# System-Assigned en VM nueva
az vm create --resource-group <rg> --name <vm> --image Ubuntu2204 \
--assign-identity --role contributor --scope /subscriptions/<id>

# System-Assigned en VM existente
```

```
az vm identity assign --resource-group <rg> --name <vm>

# User-Assigned — PASO 1: crear identidad
az identity create --resource-group <rg> --name <mi-name>

# User-Assigned — PASO 2: asignar a VM
az vm identity assign --resource-group <rg> --name <vm> \
  --identities <mi-resource-id>

# MI en App Configuration
az appconfig identity assign --name <store> --resource-group <rg>
```

Blob Storage

SDK / CÓDIGO

```
az storage account management-policy create \
  --account-name <cuenta> --policy @policy.json --resource-group <rg> # lifecycle policy
az storage container policy create \
  --account-name <cuenta> --container-name <c> \
  --name <policy> --permissions r --expiry 2026-12-31      # Stored Access Policy
```

Key Vault

SDK / CÓDIGO

```
az keyvault create --name <vault> --resource-group <rg> \
  --enable-purge-protection true --enable-soft-delete true    # crear con protección
az keyvault secret set --vault-name <vault> --name <n> --value <v> # crear secreto
az keyvault secret show --vault-name <vault> --name <n>      # leer secreto
```

TIPS FINALES PARA EL EXAMEN

Patrones que el examen frecuentemente evalúa

- Elegir el nivel de ACR correcto (geo-replicación → Premium, producción normal → Standard)
- Elegir el tier de Blob Storage (frecuente → Hot, 30d → Cool, 90d → Cold, 180d → Archive)
- Archive requiere rehidratación (horas) — Standard hasta 15h, High priority menos de 1h para <10GB
- Consistencia **Session** es el DEFAULT de Cosmos DB
- Distinguir Event Grid (reactivo/push) vs Event Hubs (streaming/Big Data) vs Service Bus (mensajería empresarial con garantías)
- **DefaultAzureCredential**: orden de autenticación y qué funciona tanto en desarrollo como en producción
- **Managed Identity**: System-Assigned = único por recurso y se elimina con él. User-Assigned = reutilizable, proceso de 2 pasos
- **App Service escalado**: scale-out usa OR para reglas de escalar, AND para reglas de reducir
- **Lifecycle policy**: si múltiples acciones aplican al mismo blob, se aplica la más económica
- **Key Vault**: Purge Protection es IRREVERSIBLE. CMK requiere Purge Protection
- **APIM**: <base /> es CRÍTICO — sin él, políticas de niveles superiores no se ejecutan
- **Durable Functions Orchestrator**: determinista — no usar DateTime.Now, Guid.NewGuid() ni I/O directo
- **Event Hubs**: número de particiones = máximo de consumidores paralelos por consumer group
- **PeekLock en Service Bus / 2-step delete en Storage Queue**: garantiza que mensajes no se pierdan ante fallos
- **Service Bus Basic tier**: SOLO Queues. Para Topics/Subscriptions → Standard o Premium
- **SAS**: User Delegation SAS es la más segura (usa Entra ID, no la clave de la cuenta)
- **HTTP Trigger Functions**: límite absoluto de 230 segundos por el load balancer, sin importar el timeout configurado
- **ACA escalado a cero**: NO disponible cuando escala por CPU o memoria
- **Application Logging**: se auto-desactiva a las 12 horas — usar Blob Storage para auditorías persistentes
- **Metadatos Blob**: PUT sobrescribe TODOS los metadatos. No hay actualizaciones parciales
- **Event Grid DLQ**: no está activa por defecto. HTTP 400/413 del handler = no reintenta
- **Partición lógica Cosmos DB**: máximo 20 GB. Partición física: 50 GB y 10,000 RU/s
- **`Azure.Storage.Blobs.Specialized`**: namespace ADICIONAL obligatorio para BlockBlobClient, AppendBlobClient y PageBlobClient
- **`CosmosClient` como Singleton**: una sola instancia durante toda la vida de la app — gestión eficiente de conexiones
- **`BlobContainerClient` directo**: puede instanciarse SIN **BlobServiceClient** si solo trabajas con un contenedor
- **User-Assigned MI — 2 pasos**: `az identity create` (crear) → `az vm identity assign` (asignar)

Errores comunes a evitar

- NO confundir Event Grid (eventos, reactivo) con Event Hubs (streaming, volumen masivo)
- NO usar Implicit flow de OAuth en apps nuevas (obsoleto — usar Auth Code + PKCE)
- NO hardcodear Connection Strings — usar App Configuration + referencias a Key Vault
- NO asumir que App Service Basic soporta escalado automático — requiere Standard o superior
- NO confundir Application Object con Service Principal — son conceptos distintos en Entra ID
- NO usar ApproximateMessagesCount de Storage Queue para lógica crítica — puede ser mayor al real
- NO usar Microsoft Graph API endpoint beta en producción
- NO olvidar que la partition key de Cosmos DB es INMUTABLE después de crear el contenedor
- NO asumir que Consumption plan de Functions soporta VNet — requiere Premium, Flex Consumption o Dedicated
- NO usar Azure AD Graph — está DEPRECADO. Usar Microsoft Graph

- NO olvidar que el número de particiones de Event Hubs es IMMUTABLE después de crear el hub
- NO pasar secretos como variables de entorno normales en ACI — usar secureValue
- NO confundir DLQ de Service Bus (sub-cola interna) con DLQ de Event Grid (Storage Container externo)
- NO olvidar que Strong consistency de Cosmos DB NO está disponible en modo multi-región
- NO usar .WithCertificate() y .WithClientSecret() juntos en MSAL — son mutuamente excluyentes
- NO olvidar el import de **Azure.Storage.Blobs.Specialized** para tipos de blob especializados
- NO instanciar múltiples CosmosClient — debe ser Singleton en toda la aplicación
- NO saltarse el PASO 1 (az identity create) al crear User-Assigned MI — es un proceso de 2 comandos

¡Buena suerte en el examen AZ-204! 🚀

Puntuación mínima para aprobar: 700 / 1000